

Pointer

- Pointer tipi değişkenler normal değişkenlerin adreslerini (başlangıç adreslerini) tutan değişkenlerdir.
- C programlama dilini güçlü kılan özelliklerden biri de pointerlerin etkin bir şekilde kullanılmasıdır.
- Pointerler kullanılarak C programlama dilinde adres bazında işlemler yapılabilmektedir.
- Özellikle dinamik bellek kullanımında veya stringler ile çalışırken, doğrudan bellek adreslerini kullanmak gerekebilecektir.

Pointer

- Çok büyük verileri kopyalamak yerine adresini göndermek
- 150 fotoğraflık albümü WhatsApp'ta paylaşıırken kopyalamıyorsun, sadece link atıyorsun

POINTERLER (İŞARETÇİLER)

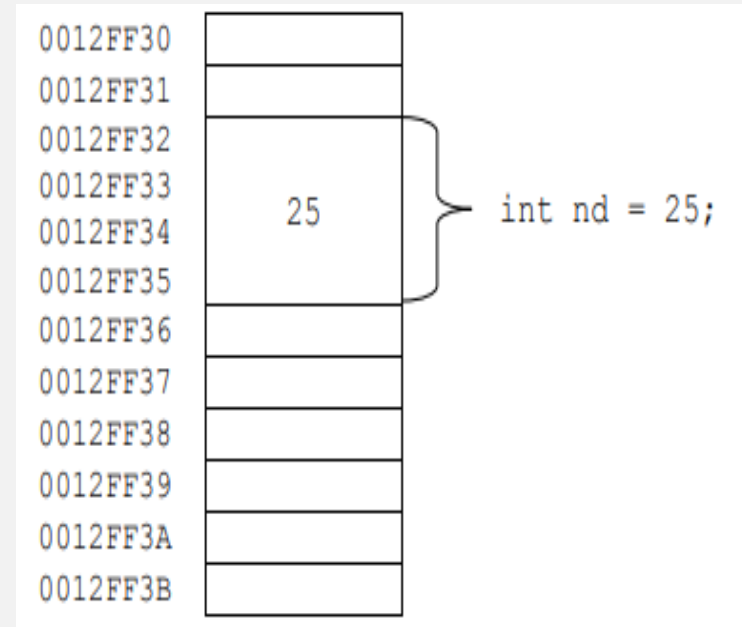
```
#include <stdio.h>
```

```
int main(void) {  
    int a = 10;    // normal değişken  
    int *p = &a;   // p, a'nın adresini tutan pointer  
  
    printf("a = %d\n", a);  
    printf("*p = %d\n", *p); // p'nin gösterdiği değeri okur (10)  
  
    *p = 25;       // pointer üzerinden a'yı değiştirir  
    printf("a = %d\n", a); // 25  
  
    return 0;  
}
```

- a → değer
- &a → adres
- p → adres tutan değişken
- *p → o adresteki değer
- p = &a; ise *p ile a aynı şeye erişir.

POINTERLER (İŞARETÇİLER)

- Programlama dillerinde bir değişken tanımlandığında, o değişkene tipine bağlı olarak RAM'dan bir hücre veya hücreler ayrılır.
- Örneğin `nd` adlı tamsayı (`int`) bir değişken tanımlayıp 25 değerini atadığımızı varsayalım. Bu değişken için bellekte aşağıdaki şekilde olduğu gibi 4 byte'lık yer ayrılır.

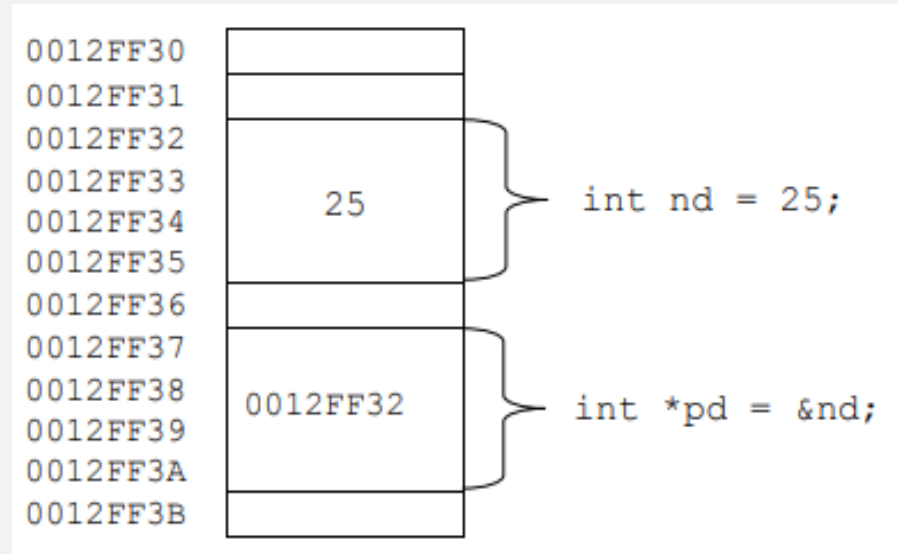


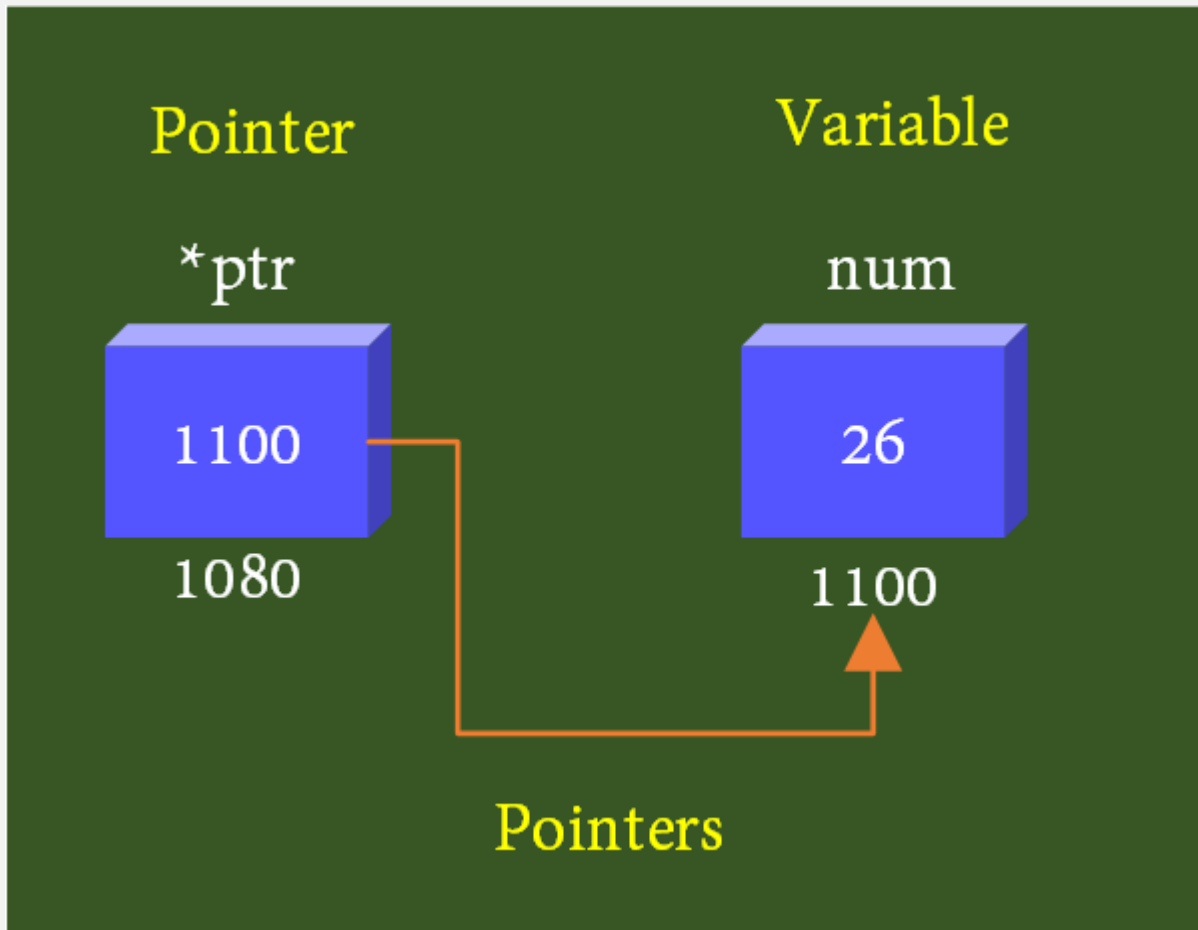
POINTERLER (İŞARETÇİLER)

- pointer, başka bir değişkenin bilgisayarın belleğindeki adresini gösteren değişkendir. Pointerler veriler yerine, bu verilerin saklı oldukları adresleri tutarlar.

POINTERLER (İŞARETÇİLER)

- normal değişkenin başlangıç adresi 0012FF32'dir. Pointer değişken, bu normal değişkenin hangi adreste bulunduğunu işaret etmektedir. Normal değişkenin bulunduğu bellek hücrelerinde ise 25 sayısı, pointer değişkenin bellek hücrelerinde ise normal değişkenin başlangıç adresi olan 0012FF32 sayısı tutulabilmektedir.





```
#include <stdio.h>
```

```
int main() {
```

```
    int yas = 20;
```

```
    int *p = &yas;
```

```
    *p = 25;
```

```
    printf("1) yas'ın değeri      : %d\n", yas);    // 25
```

```
    printf("2) p'nin içi (adres)  : %p\n", p);      // 0x7fff5e3c4b1c ← yas'ın adresi
```

```
    printf("3) *p (değer)        : %d\n", *p);      // 25
```

```
    printf("4) &p (p'nin adresi) : %p\n", &p);      // 0x7fff5e3c4b20 ← p kendi adresi
```

```
    return 0;
```

```
}
```

```
// 1A) Normal kullanım (pointer yok)
```

```
int yas = 20;
```

```
yas = 25;           // yaşı değiştirdik
```

```
printf("Yas: %d\n", yas); // → 25
```

İfade	Ne Anlama Gelir?	Karşılığı
yas	Değişkenin kendi değeri	25
&yas	Değişkenin adresi	p ile aynı
p	Pointer'ın tuttuğu adres	&yas ile aynı
*p	Pointer'ın işaret ettiği değer	yas ile aynı
&p	Pointer'ın bellekteki kendi adresi	Tamamen farklı bir adres

Programlamaya Giriş

&p

KTO Karatekin Üniversitesi

Pointer

- `int x=10, y=3;` tanımla.
- `p` pointer'ını önce `x`'e bağla: `p = &x;`
 - `x, &x, p, *p` değerlerini yazdır.
- Pointer üzerinden `x`'i 5 artır: `*p = *p + 5;`
 - tekrar `x` ve `*p` yazdır.
- `p`'yi `y`'ye bağla: `p = &y;`
 - `y, &y, p, *p` yazdır.
- Pointer üzerinden `y`'yi `x`'in değerine eşitle: `*p = x;`
 - son durumda `x` ve `y` yazdır.

```
#include <stdio.h>
```

```
int main(void) {
```

```
    int x = 10;
```

```
    int y = 3;
```

```
    int *p;
```

```
    p = &x; // p'yi önce x'e bağla ve yazdır
```

```
    printf("1) x=%d &x=%p p=%p *p=%d\n", x, &x, p, *p);
```

```
    *p = *p + 5; // artır ve yazdır
```

```
    printf("2) x=%d *p=%d\n", x, *p);
```

```
    p = &y; // değiştir ve yazdır
```

```
    printf("3) y=%d &y=%p p=%p *p=%d\n", y, &y, p, *p);
```

```
    *p = x; // 4 (y = x) ve yazdır
```

```
    printf(" SON: x=%d y=%d\n", x, y);
```

```
    return 0;
```

```
}
```

Pointer + Dizi

- `int a[5] = {10, 20, 30, 40, 50};`
- `a[0] a[1] a[2] a[3] a[4]`
`10 20 30 40 50`
- Dizi adı çoğu yerde ilk elemanın adresi gibi davranır:
`a ≈ &a[0]`
- `int *p = a; // p = &a[0] demek gibi`

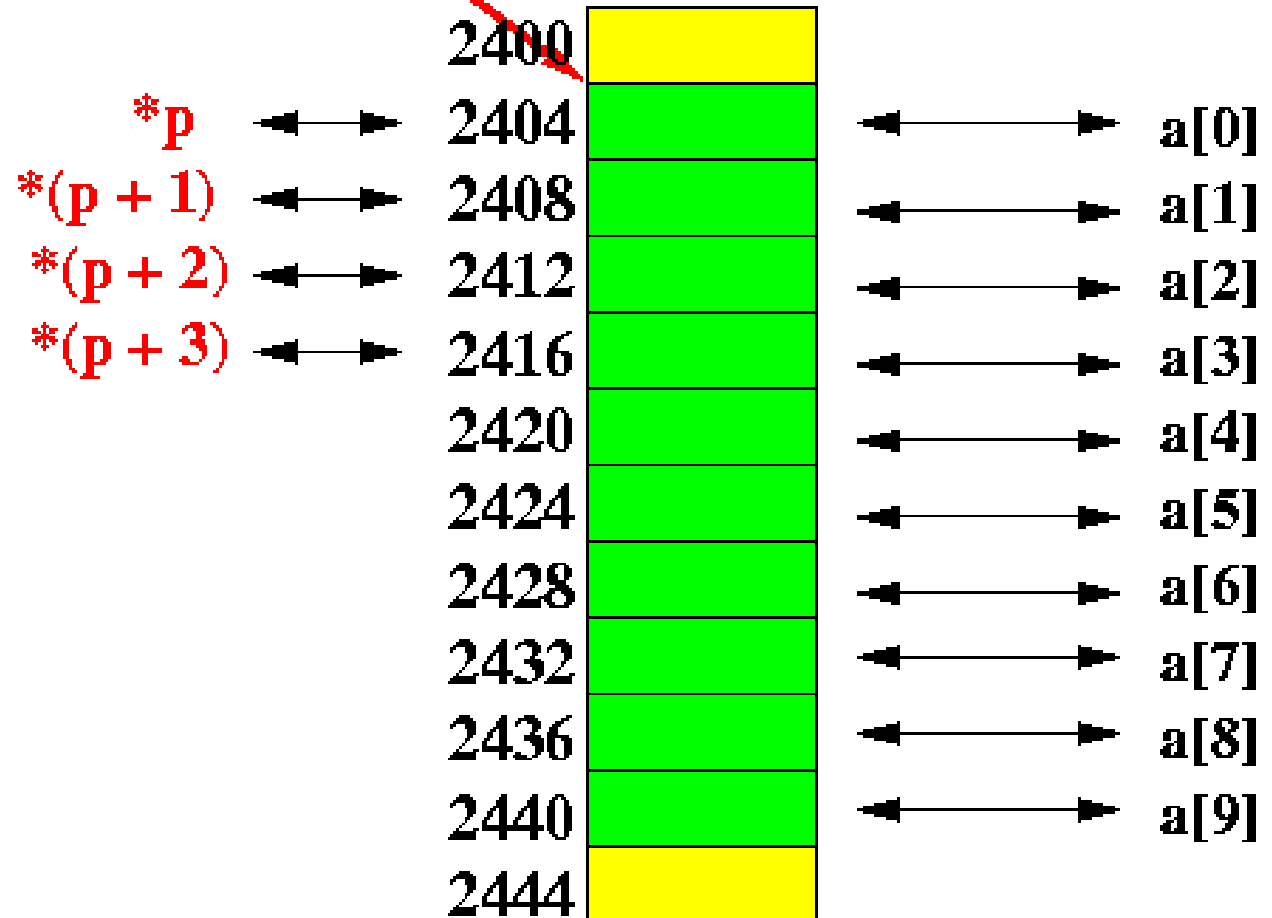
- `p` : İlk elemanın adresi (`&a[0]`)
- `p + 1` : İkinci elemanın adresi (`&a[1]`)
- `*(p + 1)` : İkinci elemanın **değeri** (yani `20`)

Not: `p + 1` dediğinde adres sadece 1 artmaz; işaret ettiği veri tipinin boyutu kadar (örneğin 4 byte) ileri atlar.

Pointer + Dizi

```
int *p;  
p = &a[0];
```

```
int a[10];
```



Pointer + Dizi

- **Dizi (a[5]):** Yan yana dizilmiş 5 vagonlu bir trendir. İçinde sayılar (10, 20, 30...) taşır.
- **Dizi Adı (a):** Trenin en başındaki lokomotifdir. Birine "Tren nerede?" diye sorduğunda, sana lokomotifin olduğu yeri (başlangıç adresini) gösterir.
- **Pointer (*p):** Lokomotifin yerini not alan bir kağıt gibidir. `int *p = a;` dediğinde, lokomotifin adresini o kağıda yazmış olursun.

Pointer + Dizi

- **$a + i$ ne demek?**
- $a \rightarrow$ ilk elemanın adresi ($\&a[0]$)
- $a + 1 \rightarrow$ bir sonraki elemanın adresi ($\&a[1]$)
- $a + i \rightarrow$ i'inci elemanın adresi ($\&a[i]$)
- Yani: **$a + i \approx \&a[i]$**
- **$*(a+i)$ ne demek?**
- $*(a+i)$ ifadesi, $a[i]$ ile aynı elemanın değerini verir:
- $a[i] == *(a + i)$

Pointer + Dizi

- Diziyi pointer ile gezmek
- Yöntem 1: ***(a+i)** ile

```
for (int i = 0; i < n; i++) {  
    printf("%d\n", *(a + i));  
}
```

*(a + i) ile a[i] tamamen aynıdır.

```
for (int i = 0; i < n; i++) {  
    // (a + i) -> i. elemana giden adres  
    // *(a + i) -> o adresteki değer (yani a[i])  
    printf("%d\n", *(a + i));  
}
```

Pointer + Dizi

- Diziyi pointer ile gezmek
- Yöntem 2: **p** pointer'ını ilerleterek

```
int *p = a; // p = &a[0]
for (int i = 0; i < n; i++) {
    printf("%d\n", *p);
    p++;    // bir sonraki elemana geç
}
```

Burada p++ demek:

- “adresin üzerine 1 ekle” değil,
- “**bir sonraki int elemanın adresine git**” demektir.

Pointer + Dizi

```
#include <stdio.h>

int main(void) {
    int a[4] = {10, 20, 30, 40};
    int *p = a;      // p -> a[0] (dizinin başlangıç adresi)
    printf("%d\n", *p); // 10
    p++;             // bir sonraki elemana geç (a[1])
    printf("%d\n", *p); // 20
    *(p + 2) = 99;    // a[3] = 99 (çünkü p şu an a[1], +2 => a[3])
    printf("%d\n", a[3]); // 99
    return 0;
}
```

Pointer + Dizi

- sayilar adında 4 elemanlı bir tamsayı dizi oluşturunuz ve başlangıç değerlerini sırasıyla {5, 12, 25, 30} yapınız.
- ptr adında bir int* pointer tanımlayınız ve dizinin başlangıcını gösterecek şekilde ayarlayınız.
- ptr pointer'ını bir adım ilerletiniz (artık sayilar[1]'i gösterecek). Pointer'ın gösterdiği elemanın değerine 8 ekleyiniz. Pointer'ın gösterdiği konumdan 2 ileri elemanı 50 yapınız.
- sayilar[1], sayilar[3], *ptr değerlerini ve Dizinin Son Halini yazdırın

```

#include <stdio.h>

int main() {
    int sayilar[] = {5, 12, 25, 30};
    int *ptr = sayilar; // Pointer dizinin başına (index 0) konumlanıyor
    printf("Dizinin İlk Hali: "); // 1. Dizinin İlk Halini Yazdırma
    for (int i = 0; i < 4; i++) {
        printf("%d ", sayilar[i]);
    }
    printf("\n-----\n");
    ptr++; // Pointer bir adım ilerliyor (Artık index 1'i gösteriyor)
    *ptr = *ptr + 8; // index 1'deki değer: 12 + 8 = 20 oluyor
    *(ptr + 2) = 50; // index 1'den 2 ileriye (index 3) 50 atanıyor
    printf("Analiz:\n");
    printf("1. Deger (sayilar[1]): %d\n", sayilar[1]);
    printf("2. Deger (sayilar[3]): %d\n", sayilar[3]);
    printf("3. Deger (*ptr): %d\n", *ptr); // ptr hala index 1'de
    printf("-----\n"); // 3. Dizinin Son Halini Yazdırma
    printf("Dizinin Son Hali: ");
    for (int i = 0; i < 4; i++) {
        printf("%d ", sayilar[i]);
    }
    printf("\n");
    return 0;
}

```

Pointer + Dizi

- Programınız sadece pointer kullanarak aşağıdakileri yapmalıdır:
- Dizinin elemanlarının toplamını hesaplayıp ekrana yazdırınız.
- Dizideki en büyük elemanı ve bu elemanın indeksini bulup yazdırınız.
 - İndeksi bulmak için pointer farkını kullanınız: $(mx - a)$
- Diziyi yerinde ters çeviriniz (ilk eleman $\leftrightarrow$ son eleman ...) ve son hali yazdırınız.

```

#include <stdio.h>
int main(void){
    int a[] = {5, 1, 9, 2};
    int n = (int)(sizeof(a) / sizeof(a[0])); int sum = 0;
    int *p = a;          // p pointer'ı dizinin başını gösterir (p = &a[0])
    while (p < a + n) {   // p, dizinin sonunu (a+n) geçene kadar dolaş
                          // a+n -> &a[n] (son elemandan 1 sonrası)
        sum += *p;        // *p: p'nin gösterdiği elemanı oku ve sum'a ekle
        p++;             // p'yi 1 eleman ileri taşı (bir sonraki dizi elemanı)
    }
    printf("sum=%d\n", sum);
    int *mx = a;          // 2) MAX için: mx başlangıçta a[0]'ı gösterebilir
    p = a + 1;            // p'yi ikinci elemana al(a[1]'den itibaren karşılaştırmak)
    while (p < a + n) {   // p dizinin sonuna kadar gezer
        if (*p > *mx) mx = p; // eğer p'nin gösterdiği değer daha büyükse,
        p++;              // bir sonraki elemana geç
    }
    printf("max=%d idx=%d\n", // max değeri ve indeksini yazdır
        *mx, (long)(mx - a)); // mx-a -> mx pointer'ı kaç eleman ileride?
                          //örn mx=&a[2] ise mx-a = 2
    int *l = a;           // 3) TERS ÇEVİRME: sol pointer (baş)
    int *r = a + (n - 1); // sağ pointer (son eleman: &a[n-1])
    while (l < r) {       // sol ve sağ pointer birbirini geçene kadar devam et
        int tmp = *l;     // sol elemanı geçici değişkene al
        *l = *r;          // sağ elemanı sola yaz
        *r = tmp;         // geçici (eski sol) değeri sağa yaz
        l++;              // solu 1 adım sağa kaydır
        r--;              // sağı 1 adım sola kaydır
    }
    printf("reversed: ");
    for (int i = 0; i < n; i++) // i=0'dan n-1'e kadar gez
        printf("%d ", a[i]);   // dizinin i. elemanını yazdır
    printf("\n");             // satır sonu
    return 0;                 // program başarıyla bitti (işletim sistemine 0 döndür)
}

```

Pointer + Fonksiyon

- C'de fonksiyonlar varsayılan olarak değer kopyası ile çalışır (pass-by-value).

Yani:

- $f(x)$ çağrısında fonksiyonun içindeki parametre, x 'in kopyasıdır.
- Fonksiyon içinde parametreyi değiştirsen bile dışarıdaki x değişmez.

Pointer + Fonksiyon

Pass-by-value (kopya gider)

```
void inc_b(int x){  
    x++;  
}
```

- $\text{inc_b}(x) \rightarrow x$ 'in kopyası artar, gerçek x aynı kalır.

Pointer + Fonksiyon

Pointer ile (adres gider)

```
void inc_ok(int *p){  
    (*p)++;  
}
```

- `inc_ok(&x)` → `x`'in adresi gider, `*p` üzerinden `x` artar.

Pointer + Fonksiyon

```
#include <stdio.h>

void ikiyleCarp(int *p) { // p: int'in adresi
    *p = (*p) * 2;        // adresteki değeri değiştir
}

int main(void) {
    int x = 7;
    ikiyleCarp(&x);        // x'in adresini gönder
    printf("%d\n", x);     // 14
    return 0;
}
```

```
#include <stdio.h>

void ikiyleCarp_ref(int *p) { // ADRESLE: değiştirir
    *p = (*p) * 2;
}

int ikiyleCarp_val(int x) { // DEĞERLE: x'in kopyası gelir, x değişmez
    x = x * 2;
    return x;
}

int main(void) {
    int x = 7;
    int y = ikiyleCarp_val(x); // x değişmez
    printf("ikiyleCarp_val: x=%d, y=%d\n", x, y); // x=7, y=14
    ikiyleCarp_ref(&x); // x değişir
    printf("ikiyleCarp_ref: x=%d\n", x); // x=14
    return 0;
}
```

Pointer + Fonksiyon

- Dizi ve Pointer: {10, 20, 30, 40} değerlerini içeren bir int dizisi ve bu diziyi gösteren bir pointer tanımlayın.
- Yazdırma: Dizi elemanlarını bir döngü içinde sadece pointer aritmetiği ($*(p + i)$) kullanarak ekrana yazdırın.
- Fonksiyon: `int *ptr`, `indis` ve `yeniDeger` parametrelerini alan, dizi elemanını pointer üzerinden güncelleyen bir fonksiyon yazın ve bu fonksiyonla dizinin son elemanını 99 yapın.
- Adres Farkı: Dizinin başı ile sonu arasındaki eleman farkını pointer çıkarma işlemiyle ekrana basın.

```
#include <stdio.h>
// Dizideki bir değeri pointer aritmetiği ile güncelleyen fonksiyon
void degeriGuncelle(int *ptr, int indis, int yeniDeger) {
    *(ptr + indis) = yeniDeger; // p[indis] ile aynı mantık
}

int main(void) {
    int a[4] = {10, 20, 30, 40};
    int *p = a; // Dizinin başlangıç adresi

    // 1. Pointer'ı döngü içinde kaydırarak yazdırma
    printf("Dizi elemanlari: ");
    for (int i = 0; i < 4; i++) {
        printf("%d ", *(p + i));
    }

    // 2. Fonksiyon kullanarak pointer üzerinden güncelleme
    // Dizinin 3. indeksini (a[3]) 99 yapalım
    degeriGuncelle(a, 3, 99);

    // 3. Pointer mesafesini ölçme
    int *son = &a[3];
    printf("\n\nSon eleman ile baslangic arasindaki fark: %ld birim\n", son - p);
    printf("Guncellenen a[3]: %d\n", a[3]);
    return 0;
}
```

C dilinde **pointer parametreleri** kullanarak aşağıdaki fonksiyonları yazınız ve verilen main fonksiyonuyla test ediniz.

İstenenler

1.inc_ok(int *p)

- p'nin gösterdiği sayıyı **1 artırmalıdır.**

2.swap_ok(int *a, int *b)

- a ve b'nin gösterdiği sayıların değerlerini **yer değiştirmelidir.**

3.divmod_int(int a, int b, int *q, int *r)

- a / b değerini *q değişkenine,
- a % b değerini *r değişkenine yazmalıdır.

(Bu fonksiyon **iki çıktıyı** pointer ile döndürmüş olacaktır.)

```

#include <stdio.h>
void inc_ok(int *p){           // inc_ok: bir int'in ADRESİNİ alır (pointer)
    // p -> artırılacak sayının adresi
    (*p)++;                   // p'nin gösterdiği değeri 1 artırır
    // *p yazarsak: adresteki değer
    // (*p)++ demek: değeri 1 artır (x değişir)
}
void swap_ok(int *a, int *b){   // swap_ok: iki sayının adresini alır
    int t = *a;                // t'ye a'nın gösterdiği değeri kopyala (geçici değişken)
    *a = *b;                    // a'nın gösterdiği yere b'nin değerini yaz
    *b = t;                     // b'nin gösterdiği yere eski a değerini yaz
}
void divmod_int(int a, int b, int *q, int *r){
    // divmod_int: a ve b değer olarak gelir (kopya)
    // q ve r ise adres olarak gelir (sonucu dışarı yazmak için)
    // Amaç: a/b bölümünü q'ya, a%b kalanını r'ye yazmak
    *q = a / b;                 // bölüm hesapla ve q'nun gösterdiği yere yaz
    *r = a % b;                 // kalan hesapla ve r'nin gösterdiği yere yaz
}
int main(void){
    int x = 10, y = 3;
    inc_ok(&x);                 // x'in ADRESİNİ gönderiyoruz
    // &x -> x'in adresi
    // inc_ok içinde (*p)++ olduğu için x gerçekten artar
    swap_ok(&x, &y);             // x ve y'nin adreslerini gönderiyoruz
    // swap_ok içinde *a ve *b üzerinden yazıldığı için
    // x ile y'nin değerleri yer değiştirir
    int q, r;                   // bölüm (q) ve kalan (r) değişkenleri
    divmod_int(17, 5, &q, &r);   // 17/5 ve 17%5 değerlerini hesaplatıyoruz
    // q ve r sonuçları dışarı "yazdırma" amaçlı out-parameter
    // Bu yüzden &q ve &r (adres) gönderiyoruz
    printf("x=%d\n", x);
    printf("y=%d\n", y);
    printf("q=%d r=%d\n", q, r); // bölüm ve kalanı yazdırır
    return 0;
}

```